Exceptions në Python

Leksion

Alion Emini

kurse.informatika
Kurs i Programimit në Python

September 29, 2025

Përmbajtja

1. Hyrje: çfarë është një exception?
2. Shmangia e gabimit me kontroll paraprak
3. Kur s'mund të shmangesh: `try/except`
4. Files dhe Exceptions
5. Shumë `except`-e dhe kapje e përgjithshme
6. `else` dhe `finally`
7. Nëse një exception s'trajtohet & `raise/assert`
8. Pasurimi me `add_note` (Py 3.11+)

Objektivat e kapitullit

- *Exception* dhe *traceback*: rreshti, moduli, lloji i gabimit.
- Shmangie me kontrolle paraprake (p.sh., `/ 0`).
- `try` / `except` ; `ValueError` , `IOError` / `FileNotFoundError` , etj.
- Shumë `except` dhe/ose `except Exception as e` .
- `except ... as err` dhe `print(err)` për mesazhin e parazgjedhur.
- `else` (rrjedhë normale) dhe `finally` (pastrim burimesh).
- `raise` ; dallimi me `assert` (çaktivizohet me `-O`).
- (Py 3.11+) `Exception.add_note(...)` për kontekst diagnostikues.

Përmbajtja

1. Hyrje: çfarë është një exception?
2. Shmangia e gabimit me kontroll paraprak
3. Kur s'mund të shmangesh: `try/except`
4. Files dhe Exceptions
5. Shumë `except`-e dhe kapje e përgjithshme
6. `else` dhe `finally`
7. Nëse një exception s'trajtohet & `raise/assert`
8. Pasurimi me `add_note` (Py 3.11+)

Përkufizim dhe ide

Përkufizim

Një **exception** është një gabim që ndodh *gjatë ekzekutimit*. Nëse nuk kapet, programi ndalet me *traceback*.

Traceback

Tregon: (1) rrugën e thirrjeve, (2) numrat e rreshtave, (3) emrin e exception-it dhe mesazhin.

Programi 1 — division1.py

```python
def main():
    num1 = int(input('Enter a number: '))
    num2 = int(input('Enter another number: '))

    result = num1 / num2
    print(f'{num1} divided by {num2} is {result}')

if __name__ == '__main__':
    main()
```

Programi 1 — Output (sukses)

Terminal

```
Enter a number:  10
Enter another number:  2
10 divided by 2 is 5.0
```

Programi 1 — Output (gabim: pjesëtim me zero)

Traceback

```
Enter a number:  10
Enter another number:  0
Traceback (most recent call last):
  File "C:\Python\division.py", line 13, in <module>
    main()
  File "C:\Python\division.py", line 9, in main
    result = num1 / num2
ZeroDivisionError:  integer division or modulo by zero
```

Çfarë tregon traceback-u?

Pika kyçe

- **Rruga e thirrjeve**: modul → funksion → rresht.
- **Lloji i gabimit**: p.sh. `ZeroDivisionError`.
- **Mesazhi**: p.sh. `integer division or modulo by zero`.

Përmbajtja

1. Hyrje: çfarë është një exception?
2. Shmangia e gabimit me kontroll paraprak
3. Kur s'mund të shmangesh: `try/except`
4. Files dhe Exceptions
5. Shumë `except`-e dhe kapje e përgjithshme
6. `else` dhe `finally`
7. Nëse një exception s'trajtohet & `raise/assert`
8. Pasurimi me `add_note` (Py 3.11+)

Parandalimi i exception-eve me kontroll paraprak

Teori

Shumë exception-e mund të parandalohen duke shkruar kod me kujdes. Në shembullin tonë, një `if` i thjeshtë mjafton që të mos lejojmë pjesëtim me zero. Në vend që ta lëmë exception-in të ngrihet, ne kontrollojmë vlerën e `num2` dhe shfaqim një mesazh gabimi kur është 0. Kjo quhet **shmangie e hijshme** (*graceful*) e një exception-i.

Programi 2 — division2.py

```python
def main():
    num1 = int(input('Enter a number: '))
    num2 = int(input('Enter another number: '))

    if num2 != 0:
        result = num1 / num2
        print(f'{num1} divided by {num2} is {result}')
    else:
        print('Cannot divide by zero.')

if __name__ == '__main__':
    main()
```

Programi 2 — Output-e

Sukses

```
Enter a number:  10
Enter another number:  2
10 divided by 2 is 5.0
```

Pjesëtim me zero

```
Enter a number:  10
Enter another number:  0
Cannot divide by zero.
```

Përmbajtja

1. Hyrje: çfarë është një exception?
2. Shmangia e gabimit me kontroll paraprak
3. Kur s'mund të shmangesh: `try/except`
4. Files dhe Exceptions
5. Shumë `except`-e dhe kapje e përgjithshme
6. `else` dhe `finally`
7. Nëse një exception s'trajtohet & `raise/assert`
8. Pasurimi me `add_note` (Py 3.11+)

Kur kontrolli paraprak nuk mjafton

Teori

Edhe kur programi është shkruar me kujdes, ka raste kur gabimet e hyrjes (*input*) nuk mund t'i parandalojmë thjesht me `if`. Shembulli i mëposhtëm llogarit pagën bruto (*gross pay*) duke kërkuar orët e punës dhe pagesën orare; nëse përdoruesi fut tekst në vend të numrit, ndodh `ValueError`.

Programi 3 — gross_pay1.py (pa trajtim)

```
1  def main():
2      hours = int(input('How many hours did you work? '))
3      pay_rate = float(input('Enter your hourly pay rate: '))
4      gross_pay = hours * pay_rate
5
6      print('Gross pay: ', gross_pay)
7
8  if __name__ == '__main__':
9      main()
```

Programi 3 — Traceback (gabim input-i)

Terminal

```
How many hours did you work?  forty
Traceback (most recent call last):
  File "...gross_pay1.py", line 17, in <module>
    main()
  File "...gross_pay1.py", line 5, in main
    hours = int(input('How many hours did you work?  '))
ValueError:  invalid literal for int() with base 10:  'forty'
```

Forma bazë: `try/except`

Struktura `try/except` përdoret për të kapur dhe trajtuar gabimet (exceptions) gjatë ekzekutimit të kodit. Kodi brenda `try` ekzekutohet normalisht; nëse ndodh një gabim, kontrolli kalon te blloku `except` për të trajtuar gabimin pa ndërprerë programin.

```
try:
    # try suite
    statement
    ...
except ExceptionName:
    # handler
    statement
    ...
```

Programi 4 — gross_pay2.py (trajtim i ValueError)

```python
def main():
    try:
        hours = int(input('How many hours did you work? '))
        pay_rate = float(input('Enter your hourly pay rate: '))
        gross_pay = hours * pay_rate

        print('Gross pay:', gross_pay)
    except ValueError:
        print('ERROR: Hours worked and hourly pay rate must be valid numbers
            .')

if __name__ == '__main__':
    main()
```

Programi 4 — Output (gabim i kapur)

Terminal

```
How many hours did you work?  forty
ERROR: Hours worked and hourly pay rate must be valid numbers.
```

Çfarë ndodh hap-pas-hapi?

- `int(input(...))` kthen `'forty'` $\longrightarrow$ `ValueError`.
- Ekzekutimi **del** nga `try` dhe **kërcehet** te `except ValueError:`.
- Rreshtat e mbetur të `try` *anashkalohen*.

Përmbajtja

1. Hyrje: çfarë është një exception?
2. Shmangia e gabimit me kontroll paraprak
3. Kur s'mund të shmangesh: `try/except`
4. Files dhe Exceptions
5. Shumë `except`-e dhe kapje e përgjithshme
6. `else` dhe `finally`
7. Nëse një exception s'trajtohet & `raise/assert`
8. Pasurimi me `add_note` (Py 3.11+)

Programi 5 — display_file.py (pa trajtim)

```python
def main():
    filename = input('Enter a filename: ')
    infile = open(filename, 'r')
    contents = infile.read()
    print(contents)
    infile.close()

if __name__ == '__main__':
    main()
```

Programi 5 — Traceback kur skedari mungon

Terminal

```
Enter a filename: bad_file.txt
Traceback (most recent call last):
  File "...display_file.py", line 21, in <module>
    main()
  File "...display_file.py", line 9, in main
    infile = open(filename, 'r')
IOError: [Errno 2] No such file or directory: 'bad_file.txt'
```

Shënim

Në Python 3 modern, ngrihet zakonisht `FileNotFoundError` (nënklasë e `OSError/IOError`).

Programi 6 — display_file2.py (trajtim i IOError)

```
def main():
    filename = input('Enter a filename: ')
    try:
        infile = open(filename, 'r')
        contents = infile.read()
        print(contents)
        infile.close()
    except IOError:
        print('An error occurred trying to read the file', filename)

if __name__ == '__main__':
    main()
```

Programi 6 — Output (gabim i kapur)

Terminal

```
Enter a filename:  bad_file.txt
An error occurred trying to read the file bad_file.txt
```

Përmbajtja

1. Hyrje: çfarë është një exception?
2. Shmangia e gabimit me kontroll paraprak
3. Kur s'mund të shmangesh: `try/except`
4. Files dhe Exceptions
5. Shumë `except`-e dhe kapje e përgjithshme
6. `else` dhe `finally`
7. Nëse një exception s'trajtohet & `raise/assert`
8. Pasurimi me `add_note` (Py 3.11+)

Programi 7 — sales_report1.py (disa except)

```python
def main():
    total = 0.0
    try:
        infile = open('sales_data.txt', 'r')
        for line in infile:
            amount = float(line)
            total += amount
        infile.close()
        print(f'{total:,.2f}')
    except IOError:
        print('An error occurred trying to read the file.')
    except ValueError:
        print('Non-numeric data found in the file.')
    except:
        print('An error occurred.')

if __name__ == '__main__':
    main()
```

sales_data.txt — shembull përmbajtjeje

Skedari

```
24987.62
26978.97
32589.45
31978.47
22781.76
29871.44
```

Kujtesë

`open()` mund të ngrejë `IOError`; `float(line)` mund të ngrejë `ValueError`.

Programi 8 — sales_report2.py (kap *çdo* gabim)

```
def main():
    total = 0.0
    try:
        infile = open('sales_data.txt', 'r')
        for line in infile:
            amount = float(line)
            total += amount
        infile.close()
        print(f'{total:,.2f}')
    except:
        print('An error occurred.')

if __name__ == '__main__':
    main()
```

Programi 8 — Output-e shembull

Gabim

```
Enter a filename:  bad_file.txt
An error occurred.
```

Sukses

```
Enter a filename:  sales_data.txt
24987.62
26978.97
32589.45
31978.47
22781.76
29871.44
```

Këshillë

Kapja e përgjithshme është e thjeshtë por mund të fshehë problemet reale.

Programi 9 — gross_pay3.py (as err & mesazhi i parazgjedhur)

```
def main():
    try:
        hours = int(input('How many hours did you work? '))
        pay_rate = float(input('Enter your hourly pay rate: '))
        gross_pay = hours * pay_rate
        print('Gross pay: ', gross_pay)
    except ValueError as err:
        print(err)

if __name__ == '__main__':
    main()
```

Programi 9 — Output (gabim i kapur me mesazh default)

Terminal

```
How many hours did you work?  forty
invalid literal for int() with base 10:  'forty'
```

Programi 10 — sales_report3.py (except Exception as err)

```
def main():
    total = 0.0
    try:
        infile = open('sales_data.txt', 'r')
        for line in infile:
            amount = float(line)
            total += amount
        infile.close()
        print(f'{total:,.2f}')
    except Exception as err:
        print(err)

if __name__ == '__main__':
    main()
```

Programi 10 — Output-e

Gabim

```
Enter a filename:  bad_file.txt
FileNotFoundError:  [Errno 2] No
such file or directory:
'bad_file.txt'
```

Sukses

```
Enter a filename:  sales_data.txt
169187.71
```

Përmbajtja

1. Hyrje: çfarë është një exception?
2. Shmangia e gabimit me kontroll paraprak
3. Kur s'mund të shmangesh: `try/except`
4. Files dhe Exceptions
5. Shumë `except`-e dhe kapje e përgjithshme
6. `else` dhe `finally`
7. Nëse një exception s'trajtohet & `raise/assert`
8. Pasurimi me `add_note` (Py 3.11+)

Sintaksa else

Blloku else ekzekutohet vetëm nëse nuk ndodh asnjë përjashtim në bllokun try.

```
try:
    statement
except ExceptionName:
    handler
else:
    # Ekzekutohet vetëm nëse try s'ngre gabim
    success_path
```

Programi 11 — sales_report4.py (else për rrjedhën normale)

```python
def main():
    total = 0.0
    try:
        infile = open('sales_data.txt', 'r')
        for line in infile:
            amount = float(line)
            total += amount
        infile.close()
    except Exception as err:
        print(err)
    else:
        print(f'{total:,.2f}')

if __name__ == '__main__':
    main()
```

Programi 11 — Output-e

Gabim

```
Enter a filename:  bad_file.txt
FileNotFoundError:  [Errno 2] No
such file or directory:
'bad_file.txt'
```

Sukses

```
Enter a filename:  sales_data.txt
169187.71
```

Sintaksa finally

Blloku finally ekzekutohet gjithmonë, pavarësisht nëse ndodhi një përjashtim apo jo. Kjo është e dobishme për pastrimin e burimeve, siç janë skedarët dhe lidhjet rrjeti.

```
try:
    statement
except ExceptionName:
    handler
finally:
    # Ekzekutohet GJITHMONË (edhe me gabim)
    cleanup
```

Programi 12 — sales_report5.py (finally për pastrim)

```python
def main():
    total = 0.0
    infile = None
    try:
        infile = open('sales_data.txt', 'r')
        for line in infile:
            amount = float(line)
            total += amount
    except Exception as err:
        print(err)
    else:
        print(f'{total:,.2f}')
    finally:
        print('Executing finally clause.')
        if infile is not None:
            infile.close()

if __name__ == '__main__':
    main()
```

Programi 12 — Output-e

Gabim

```
Enter a filename:  bad_file.txt
FileNotFoundError:  [Errno 2] ...
Executing finally clause.
```

Sukses

```
Enter a filename:  sales_data.txt
169187.71
Executing finally clause.
```

Përmbajtja

1. Hyrje: çfarë është një exception?
2. Shmangia e gabimit me kontroll paraprak
3. Kur s'mund të shmangesh: `try/except`
4. Files dhe Exceptions
5. Shumë `except`-e dhe kapje e përgjithshme
6. `else` dhe `finally`
7. Nëse një exception s'trajtohet & `raise/assert`
8. Pasurimi me `add_note` (Py 3.11+)

Kur një gabim s'kapet

Pasojat

- Programi ndalet me *traceback*.
- Arsye tipike: lloji i gabimit s'përputhet me asnjë `except`, ose ndodh jashtë `try`.

Si të dish çfarë të kapësh?

- Lexo dokumentimin e Python-it për llojet e exception-eve.
- Eksperimentim i qëllimshëm: provoko gabime dhe lexo *traceback*-un.

Programi 13 — low.py (raise me mesazh)

```python
def main():
    number = 10
    if number > 5:
        raise Exception(f"The number should not exceed 5. ({number=})")
    print(number)

if __name__ == "__main__":
    main()
```

Programi 13 — Traceback

Terminal

```
Traceback (most recent call last):
  File "./low.py", line 4, in <module>
    raise Exception(f"The number should not exceed 5.  (number=)")
Exception:  The number should not exceed 5.  (number=10)
```

Shënim

Mesazhi me `\{number=\}` tregon emrin dhe vlerën, duke ndihmuar diagnostikimin.

raise me lloj specifik (TypeError)

```
value = "10"

if not isinstance(value, int):
    raise TypeError(f"Expected int for {value=}, got {type(value).__name__}"
        )
```

Traceback i TypeError

Terminal

```
Traceback (most recent call last):
  File "./example.py", line 4, in <module>
    raise TypeError(...)
TypeError:  Expected int for value='10', got str
```

Këshillë

Prefero lloje specifike (ValueError, TypeError, ...) ndaj Exception.

`assert` vs `raise`

Dallimi

`assert` është për *debug*; çaktivizohet me `python -O` . Mos e përdor për logjikë prodhimi. `raise` sinjalizon gjendje të pavlefshme në runtime.

Përmbajtja

1. Hyrje: çfarë është një exception?
2. Shmangia e gabimit me kontroll paraprak
3. Kur s'mund të shmangesh: `try/except`
4. Files dhe Exceptions
5. Shumë `except`-e dhe kapje e përgjithshme
6. `else` dhe `finally`
7. Nëse një exception s'trajtohet & `raise/assert`
8. Pasurimi me `add_note` (Py 3.11+)

Shembull bazë: add_note

```
try:
    raise TypeError('bad type')
except Exception as e:
    e.add_note('Add some information')
    e.add_note('Add some more information')
    raise
```

Output i `add_note`

Traceback (shkurt)

```
TypeError:  bad type
Add some information
Add some more information
```

Kërkesë versioni

`Exception.add_note(...)` kërkon Python 3.11+.

Programi 14 — shembull.py (shënime me rreshtin/problem-in) I

```python
def process_file(filename):
    try:
        with open(filename) as f:
            for lineno, line in enumerate(f, 1):
                try:
                    value = float(line)
                    # Përpunim i mëtejshëm...
                except ValueError as err:
                    err.add_note(f"Gabim në rreshtin {lineno} të skedarit '{filename}': {line.
                        strip()}")
                    raise
    except Exception as e:
        e.add_note(f"Ndodhi gabim gjatë përpunimit të skedarit '{filename}'")
        raise

if __name__ == "__main__":
    process_file("shembull.txt")
```

Programi 14 — shembull.py (shënime me rreshtin/problem-in) II

shembull.txt

```
12.5
abc
42.0
```

Output i gabimit

```
ValueError: could not convert string to float: 'abc'
Gabim në rreshtin 2 të skedarit 'shembull.txt': abc
```

... (pas rihodhjes)

```
Ndodhi gabim gjatë përpunimit të skedarit 'shembull.txt'
```

Përmbledhje e kapitullit

- **Exception** = gabim gjatë ekzekutimit; *traceback* jep thirrjet/rreshtin/llojin/mesazhin.
- Parandalimi me kontrolle paraprake (p.sh. shmang `/ 0`).
- `try` / `except`: kap gabimet pa rrëzuar programin; kap lloje specifike.
- `except Exception as err` për kapje të përgjithshme; `print(err)` shfaq mesazhin default.
- `else` për rrjedhën e suksesshme; `finally` për pastrim (*always-run*).
- `raise` sinjalizon gjendje të pavlefshme; prefero lloje specifike.
- `assert` është për *debug* (çaktivizohet me `-O`).
- `Exception.add_note()` pasuron *traceback*-un me kontekst.
- Me skedarë: trajto gabimet I/O dhe prefero `with open(...)` për mbyllje automatike.

Faleminderit!

Pyetje?